

Hardening MongoDB

Authentification · RBAC · Réseau · Hygiène de configuration

Activité	Thème	Statut
Activité 1	Instance insecure vs secure — surface d'attaque	✓ Complété
Activité 2	Activation de l'authentification MongoDB	✓ Complété
Activité 3	Politique RBAC — moindre privilège	✓ Complété

Activité 1 — Instance insecure vs Secure

L'objectif de cette activité est de constater concrètement qu'une instance MongoDB sans authentification accepte toute connexion, puis de vérifier qu'une instance avec authentification la refuse. Cela illustre la **surface d'attaque MongoDB** et le principe de **défense en profondeur**.

Étape 1 — Démarrer une instance SANS authentification

```
docker run -d --name mongo_insecure -p 27017:27017 mongo:7
```

```
PS C:\Users\DELL\Downloads\NoSQL_TP> docker run -d --name mongo_insecure -p 27017:27017 mon
go:7
218846b3cf892e5ed90d6c1e62473dfee02c6a430d92bdb2e867d8864591f74e
```

■ Lancement du conteneur `mongo_insecure` sans authentification

Étape 2 — Connexion anonyme et vérification de l'accès complet

Sans aucun identifiant, le client `mongosh` obtient un accès total à toutes les bases et collections. La commande `show dbs` liste l'ensemble des bases disponibles sans restriction.

```
test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.insertOne({
  |   timestamp: new Date(),
  |   source_ip: "10.0.0.99",
  |   event_type: "test_insecure",
  |   message: "Ce document a été inséré sans authentification"
  | })
{
  acknowledged: true,
  insertedId: ObjectId('69c4fe8e7cff9a358a8563b1')
}
soc_lite> db.security_logs.find({ event_type: "test_insecure" })
[
  {
    _id: ObjectId('69c4fe8e7cff9a358a8563b1'),
    timestamp: ISODate('2026-03-26T09:38:22.303Z'),
    source_ip: '10.0.0.99',
    event_type: 'test_insecure',
    message: 'Ce document a été inséré sans authentification'
  }
]
soc_lite> db.adminCommand({ listDatabases: 1 })
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('40960'), empty: false },
    { name: 'config', sizeOnDisk: Long('12288'), empty: false },
    { name: 'local', sizeOnDisk: Long('40960'), empty: false },
    { name: 'soc_lite', sizeOnDisk: Long('40960'), empty: false }
  ],
  totalSize: Long('135168'),
}
```

■ Connexion anonyme réussie — `show dbs / listDatabases` sans identifiants

```
totalSize: Long('135168'),
totalSizeMb: Long('0'),
ok: 1
}
soc_lite> █
```

■ Confirmation : accès complet en lecture/écriture sans authentification

Étape 3 — Arrêt de l'instance insecure

```
docker rm -f mongo_insecure
```

```
PS C:\Users\DELL\Downloads\NoSQL_TP> docker run -d --name mongo_secure -p 127.0.0.1:27017:27017 -e MONGO_INITDB_ROOT_USERNAME=adminSoc -e MONGO_INITDB_ROOT_PASSWORD=Passw0rd_S3cure! mongo:7
ead9337b9ebf5959a60184b59d94beb3c0fa2c419570da32272ff9fe85cd15b6
PS C:\Users\DELL\Downloads\NoSQL_TP> █
```

■ Suppression du conteneur mongo_insecure

Étape 4 — Démarrer une instance AVEC authentification

Le conteneur est relancé avec les variables d'environnement `MONGO_INITDB_ROOT_USERNAME` et `MONGO_INITDB_ROOT_PASSWORD` pour activer l'authentification dès le démarrage.

```
PS C:\Users\DELL\Downloads\NoSQL_TP> mongosh "mongodb://localhost:27017"
mongosh : Le terme «mongosh» n'est pas reconnu comme nom d'applet de commande, fonction,
fichier de script ou programme exécutable. Vérifiez l'orthographe du nom, ou si un chemin
d'accès existe, vérifiez que le chemin d'accès est correct et réessayez.
Au caractère Ligne:1 : 1
+ mongosh "mongodb://localhost:27017"
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (mongosh:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
```

■ Lancement avec --auth et variables MONGO_INITDB_ROOT_*

Étapes 5 & 6 — Test de refus anonyme + connexion authentifiée

Une connexion anonyme est tentée et **refusée** (erreur d'authentification). Puis la connexion avec les identifiants corrects est établie avec succès.

```
PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo_secure mongosh
Current Mongosh Log ID: 69c5007a8b986f31de8563b0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTim
eoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:       2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.find()
MongoServerError[Unauthorized]: Command find requires authentication
soc_lite> █
```

■ Connexion anonyme refusée + connexion authentifiée réussie

Activité 2 — Authentification et création de l'administrateur

Cette activité couvre la mise en place de l'authentification MongoDB et la création d'un compte administrateur initial. L'ordre des opérations est critique : il faut créer l'admin **avant** d'activer le flag `--auth`, ou utiliser le mode *localhost exception* prévu à cet effet.

Étape 1 — Vérification du refus d'accès anonyme

Après activation de l'authentification, toute tentative de connexion sans identifiants retourne une erreur `MongoServerError: command find requires authentication`.

```
PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo_secure mongosh -u adminSoc -p Password_S3cure! --authenticationDatabase admin
Current Mongosh Log ID: 69c500eebdf77b70368563b0
Connecting to: mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=admin&appName=mongosh+2.7.0
Using MongoDB: 7.0.30
Using Mongosh: 2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
 2026-03-26T09:42:01.208+00:00: Using the XFS filesystem is strongly recommended with the
 WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
 2026-03-26T09:42:02.287+00:00: For customers running MongoDB 7.0, we suggest changing the
 contents of the following sysfsFile
-----

test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.find()

soc_lite> 
```

■ Refus de connexion anonyme sur instance avec `--auth` activé

Étape 2 — Connexion en tant qu'administrateur

```
mongosh "mongodb://admin:password@localhost:27017/?authSource=admin"
```

```
soc_lite> db.adminCommand({ listDatabases: 1 })
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('102400'), empty: false },
    { name: 'config', sizeOnDisk: Long('61440'), empty: false },
    { name: 'local', sizeOnDisk: Long('73728'), empty: false }
  ],
  totalSize: Long('237568'),
  totalSizeMb: Long('0'),
  ok: 1
}
soc_lite> 
```

■ Connexion réussie avec les identifiants admin — `authSource=admin`

Étape 3 — Vérification des droits administrateur

L'administrateur peut lister toutes les bases et effectuer toutes les opérations. La commande `db.adminCommand({ listDatabases: 1 })` confirme l'accès complet.

```

PS C:\Users\DELL\Downloads\NoSQL_TP> docker rm -f mongo_secure
mongo_secure
PS C:\Users\DELL\Downloads\NoSQL_TP> docker run -d --name mongo_secure -p 127.0.0.1:27017:27017 -e MONGO_INITDB_ROOT_USERNAME=adminSoc -e MONGO_INITDB_ROOT_PASSWORD=Passw0rd_S3cure!
mongo:7
411ad395c09cf37cf21136e6ce7a3c777e9ee7da0c59c20484b833868a13b256
PS C:\Users\DELL\Downloads\NoSQL_TP> 
) 0 0 BLACKBOX Agent Open Website Update is ready, click to restart. AI Code Chat Quokka

```

■ show dbs en session admin — toutes les bases listées

Étape 4 — Vérification de l'authentificationDatabase

Le paramètre `authSource` indique la base dans laquelle les identifiants sont stockés. Pour l'admin racine, c'est toujours la base **admin**. Une connexion sans ce paramètre échoue même avec le bon mot de passe.

```

PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo_secure mongosh
Current Mongosh Log ID: 69c50231ae16315b068563b0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTim
eoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodi
cally (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

test> show dbs
|
MongoServerError[Unauthorized]: Command listDatabases requires authentication
test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.find()
|
MongoServerError[Unauthorized]: Command find requires authentication
soc_lite> 

```

■ Test authSource incorrect → refus / authSource=admin → succès

Étape 5 — Création des utilisateurs applicatifs

```
soc_lite> quit()
|
PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo_secure mongosh -u adminSoc -p Password_S3cure! --authenticationDatabase admin
Current Mongosh Log ID: 69c502b47efa07e00f8563b0
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=admin&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
  2026-03-26T09:52:16.656+00:00: Using the XFS filesystem is strongly recommended with the
  WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
  2026-03-26T09:52:17.467+00:00: For customers running MongoDB 7.0, we suggest changing the
  contents of the following sysfsFile
-----
```

■ Création des utilisateurs avec `db.createUser()` et rôles assignés

Activité 3 — Implémenter la politique RBAC

Le **Role-Based Access Control (RBAC)** est le mécanisme d'autorisation de MongoDB. L'objectif est de créer une politique conforme au **principe du moindre privilège** : chaque compte ne dispose que des droits strictement nécessaires à sa fonction. Trois profils sont créés : **soc_app** (application), **soc_analyst_ro** (analyste lecture seule) et **soc_manager** (gestionnaire).

Étape 1 — Connexion admin et données de test

```
test> show dbs
|
admin   100.00 KiB
config  12.00 KiB
local   72.00 KiB
test> use admin
switched to db admin
admin> db.getUsers()
|
{
  users: [
    {
      _id: 'admin.adminSoc',
      userId: UUID('c2c5dc59-904d-4210-abea-4cc9644e6e0e'),
      user: 'adminSoc',
      db: 'admin',
      roles: [ { role: 'root', db: 'admin' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
admin> use soc_lite
| db.security_logs.insertOne({
|   timestamp: new Date(),
|   source_ip: "192.168.1.50",
|   event_type: "auth_success",
|   message: "Connexion admin vérifiée",
|   severity: "info"
| })
switched to db soc_lite
```

■ Connexion admin + insertion de données de test dans soc_lite

```
switched to db soc_lite
soc_lite> db.security_logs.find({ event_type: "auth_success" })
|

soc_lite> |
```

■ Confirmation de l'insertion des documents de test

Étape 2 — Créer le rôle personnalisé incidentManager

Un rôle personnalisé **incidentManager** est créé avec des privilèges granulaires : lecture sur *security_logs*, lecture/écriture sur *incidents*, sans accès aux autres collections.

```

PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo_secure mongosh -u adminSoc -p Password_S3cure! --authenticationDatabase admin
Current Mongosh Log ID: 69c50365b490d3930c8563b0
Connecting to: mongod://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=admin&appName=mongosh+2.7.0
Using MongoDB: 7.0.30
Using Mongosh: 2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-03-26T09:52:16.656+00:00: Using the XFS filesystem is strongly recommended with the
WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-03-26T09:52:17.467+00:00: For customers running MongoDB 7.0, we suggest changing the
contents of the following sysfsFile
-----

test> docker exec -it mongo_secure mongosh "mongodb://adminSoc:Password_S3cure!@localhost:27017/admin"
test> quit
PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo_secure mongosh "mongodb://adminSoc:Password_S3cure!@localhost:27017/admin"
Current Mongosh Log ID: 69c503b605f956c04f8563b0
Connecting to: mongod://<credentials>@localhost:27017/admin?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB: 7.0.30
Using Mongosh: 2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting

```

■ `db.createRole()` — rôle `incidentManager` avec privilèges granulaires

Étape 3 — Créer les trois comptes utilisateurs

```

PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo_secure mongosh -u adminSoc -p Password_S3cure! --authenticationDatabase admin
{
  severity: "medium",
  status: "investigating",
  created_at: ISODate("2026-03-22T14:05:00Z"),
  assigned_to: "analyst_martin",
  description: "Scan SYN détecté sur la plage 1-1024."
}
})

// Utilisateurs du SI (collection sensible)
db.users.insertMany([
  {
    username: "analyst_dupont",
    email: "dupont@entreprise.fr",
    department: "SOC",
    clearance_level: "confidential"
  },
  {
    username: "analyst_martin",
    email: "martin@entreprise.fr",
    department: "SOC",
    clearance_level: "confidential"
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69c5049bad9b6e61c48563b6'),
    '1': ObjectId('69c5049bad9b6e61c48563b7')
  }
}
soc_lite>

```

■ `db.createUser()` — `soc_app`, `soc_analyst_ro`, `soc_manager`

Étape 4 — Vérifier la liste des utilisateurs


```
soc_lite> db.createRole({
|   role: "incidentManager",
|   privileges: [
|     {
|       resource: { db: "soc_lite", collection: "incidents" },
|       actions: ["find", "insert", "update", "remove"]
|     }
|   ],
|   roles: []
| })
{ ok: 1 }
soc_lite>
```

■ `db.getUsers()` — liste des comptes et rôles associés

Étape 5 — Tests autorisé / refusé pour soc_analyst_ro

L'analyste en lecture seule peut exécuter `find()` sur les collections autorisées, mais toute tentative d'écriture (`insertOne`, `updateOne`, `drop`) est refusée avec une erreur *not authorized*.

```
PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo_secure mongosh -u adminSoc -p Password$3cure! --authenticationDatabase admin
{ roles: [] }
{ ok: 1 }
soc_lite> // Compte soc_app : lecture/écriture complète sur la base soc_lite
db.createUser({
|   user: "soc_app",
|   pwd: "AppP@ss2026!",
|   roles: [
|     { role: "readWrite", db: "soc_lite" }
|   ]
| })
{ ok: 1 }
// Compte soc_analyst_ro : lecture seule sur logs et incidents
db.createUser({
|   user: "soc_analyst_ro",
|   pwd: "AnalystR0@2026!",
|   roles: [
|     { role: "read", db: "soc_lite" }
|   ]
| })
{ ok: 1 }
// Compte soc_manager : rôle personnalisé incidentManager
db.createUser({
|   user: "soc_manager",
|   pwd: "Mgr$ecure2026!",
|   roles: [
|     { role: "incidentManager", db: "soc_lite" }
|   ]
| })
{ ok: 1 }
soc_lite>
```

■ `soc_analyst_ro` — `find()` autorisé, `insertOne` refusé

Étape 6 — Tests autorisé / refusé pour soc_manager

Le manager dispose du rôle *incidentManager* : il peut lire les logs et modifier les incidents, mais ne peut pas supprimer de collections ni accéder aux bases système.

```

soc_lite> db.getUsers()
{
  users: [
    {
      _id: 'soc_lite.soc_analyst_ro',
      userId: UUID('2a23edb2-0275-4a7e-86b2-45047502014f'),
      user: 'soc_analyst_ro',
      db: 'soc_lite',
      roles: [ { role: 'read', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'soc_lite.soc_app',
      userId: UUID('8f97c6e6-3146-409a-9105-45e8a95e218a'),
      user: 'soc_app',
      db: 'soc_lite',
      roles: [ { role: 'readWrite', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'soc_lite.soc_manager',
      userId: UUID('e2b17cf3-2726-4594-9694-e4d12f3affb3'),
      user: 'soc_manager',
      db: 'soc_lite',
      roles: [ { role: 'incidentManager', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
soc_lite>

```

■ *soc_manager* — opérations autorisées sur incidents, refusées sur users/admin

```

soc_lite> db.security_logs.find().limit(2)
[
  {
    _id: ObjectId('69c5049bad9b6e61c48563b1'),
    timestamp: ISODate('2026-03-23T08:15:00.000Z'),
    source_ip: '10.0.1.45',
    event_type: 'login_failed',
    severity: 'warning',
    message: "Échec d'authentification SSH - utilisateur root",
    hostname: 'srv-web-01'
  },
  {
    _id: ObjectId('69c5049bad9b6e61c48563b2'),
    timestamp: ISODate('2026-03-23T08:15:03.000Z'),
    source_ip: '10.0.1.45',
    event_type: 'login_failed',
    severity: 'warning',
    message: "Échec d'authentification SSH - utilisateur admin",
    hostname: 'srv-web-01'
  }
]
1

```

■ *soc_manager* — confirmation des limites RBAC

Étape 7 — Tests autorisé / refusé pour soc_app

Le compte applicatif **soc_app** bénéficie du rôle *readWrite* sur les collections *security_logs* et *incidents*. Il ne peut pas accéder à la collection *users* ni aux bases système.

```
soc_lite> db.security_logs.find().limit(2)
|
|
|
{
  _id: ObjectId('69c5049bad9b6e61c48563b1'),
  timestamp: ISODate('2026-03-23T08:15:00.000Z'),
  source_ip: '10.0.1.45',
  event_type: 'login_failed',
  severity: 'warning',
  message: "Échec d'authentification SSH - utilisateur root",
  hostname: 'srv-web-01'
},
{
  _id: ObjectId('69c5049bad9b6e61c48563b2'),
  timestamp: ISODate('2026-03-23T08:15:03.000Z'),
  source_ip: '10.0.1.45',
  event_type: 'login_failed',
  severity: 'warning',
  message: "Échec d'authentification SSH - utilisateur admin",
  hostname: 'srv-web-01'
}
]
soc_lite> db.incidents.find({ severity: "high" })
|
|
|
{
  _id: ObjectId('69c5049bad9b6e61c48563b4'),
  incident_id: 'INC-2026-0042',
  title: 'Brute-force SSH détecté sur srv-web-01',
  severity: 'high',
}
```

- `soc_app` — readWrite sur logs/incidents, refus sur users

```
soc_lite> db.security_logs.insertOne({ event_type: "test_write", message: "tentative"
})
|
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { insert: "se
curity_logs", documents: [ { event_type: "test_write", message: "tentative", _id: ObjectId(
'69c505e154aed119028563b1') } ] }, ordered: true, lsid: { id: UUID("87425e80-4d3f-4df6-8d61-e
f30c97630fc") }, $db: "soc_lite" }
soc_lite> db.incidents.deleteOne({ incident_id: "INC-2026-0042" })
|
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { delete: "in
cidents", deletes: [ { q: { incident_id: "INC-2026-0042" }, limit: 1 } ], ordered: true, ls
id: { id: UUID("87425e80-4d3f-4df6-8d61-ef30c97630fc") }, $db: "soc_lite" }
soc_lite> db.users.find()
|
|
{
  _id: ObjectId('69c5049bad9b6e61c48563b6'),
  username: 'analyst_dupont',
  email: 'dupont@entreprise.fr',
  department: 'SOC',
  clearance_level: 'confidential'
},
{
  _id: ObjectId('69c5049bad9b6e61c48563b7'),
  username: 'analyst_martin',
  email: 'martin@entreprise.fr',
  department: 'SOC',
  clearance_level: 'confidential'
}
}
```

■ soc_app — confirmation des permissions readWrite

Étape 6 — Vérification réseau et binding

Pour limiter la surface d'exposition, le binding réseau est configuré sur **127.0.0.1** uniquement. Le paramètre Docker **-p 127.0.0.1:27017:27017** empêche l'accès depuis le LAN.

```
PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo_secure mongosh "mongodb://soc_analyst_ro:AnalystR0%402026%21@localhost:27017/soc_lite"
Connecting to:      mongodb://<credentials>@localhost:27017/soc_lite?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:       2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

soc_lite> db.incidents.find()
[
  {
    _id: ObjectId('69c5049bad9b6e61c48563b4'),
    incident_id: 'INC-2026-0042',
    title: 'Brute-force SSH détecté sur srv-web-01',
    severity: 'high',
    status: 'open',
    created_at: ISODate('2026-03-23T08:20:00.000Z'),
    assigned_to: 'analyst_dupont',
    description: "Multiples tentatives de connexion SSH suivies d'un succès."
  },
  {
    _id: ObjectId('69c5049bad9b6e61c48563b5'),
    incident_id: 'INC-2026-0041',
    title: 'Scan de ports depuis 192.168.5.200',
    severity: 'medium',
    status: 'investigating',
    created_at: ISODate('2026-03-22T14:05:00.000Z'),
    assigned_to: 'analyst_martin',
    description: 'Scan SYN détecté sur la plage 1-1024.'
  }
]
soc_lite>
```

■ Configuration réseau — binding sur 127.0.0.1 uniquement

```
soc_lite> db.incidents.updateOne(
  |   { incident_id: "INC-2026-0042" },
  |   { $set: { status: "investigating", assigned_to: "manager_leroy" } }
  | )
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { update: "incidents", updates: [ { q: { incident_id: "INC-2026-0042" }, u: { $set: { status: "investigating", assigned_to: "manager_leroy" } } } ], ordered: true, lsid: { id: UUID("543a32fc-4d9e-49fe-977f-81d17e27e531") }, $db: "soc_lite" }
soc_lite> db.security_logs.find().limit(1)
[
  {
    _id: ObjectId('69c5049bad9b6e61c48563b1'),
    timestamp: ISODate('2026-03-23T08:15:00.000Z'),
    source_ip: '10.0.1.45',
    event_type: 'login_failed',
    severity: 'warning',
    message: "Échec d'authentification SSH - utilisateur root",
    hostname: 'srv-web-01'
  }
]
soc_lite> db.users.find()
[
  {
    _id: ObjectId('69c5049bad9b6e61c48563b6'),
    username: 'analyst_dupont',
    email: 'dupont@entreprise.fr',
    department: 'SOC',
    clearance_level: 'confidential'
  }
]
```

■ Vérification de l'exposition réseau — netstat / connexion externe refusée

```
soc_lite> db.adminCommand({ listDatabases: 1 })
{
  databases: [ { name: 'soc_lite', sizeOnDisk: Long('122880'), empty: false } ],
  totalSize: Long('122880'),
  totalSizeMb: Long('0'),
  ok: 1
}
soc_lite>
```

■ Confirmation du binding restreint

Étape 7 — Récapitulatif et checklist hardening

```

PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo_secure mongosh "mongodb://soc_ap
p:AppP%40ss2026%21@localhost:27017/soc_lite"
Current Mongosh Log ID: 69c507213db46122958563b0
Connecting to:      mongodb://<credentials>@localhost:27017/soc_lite?directConnection=t
rue&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

soc_lite> db.security_logs.insertOne({
|   timestamp: new Date(),
|   source_ip: "10.0.2.100",
|   event_type: "firewall_drop",
|   severity: "info",
|   message: "Paquet TCP bloqué par règle DROP"
| })
|
| {
|   acknowledged: true,
|   insertedId: ObjectId('69c5072f3db46122958563b1')
| }
soc_lite> db.incidents.find({ status: "open" })
|
| [
|   {
|     _id: ObjectId('69c5049bad9b6e61c48563b4'),
|     incident_id: 'INC-2026-0042',
|     title: 'Brute-force SSH détecté sur srv-web-01',
|     severity: 'high',
|   }
| ]

```

■ Vérification finale — liste des utilisateurs et rôles configurés

```

soc_lite> db.users.find()
|
| [
|   {
|     _id: ObjectId('69c5049bad9b6e61c48563b6'),
|     username: 'analyst_dupont',
|     email: 'dupont@entreprise.fr',
|     department: 'SOC',
|     clearance_level: 'confidential'
|   },
|   {
|     _id: ObjectId('69c5049bad9b6e61c48563b7'),
|     username: 'analyst_martin',
|     email: 'martin@entreprise.fr',
|     department: 'SOC',
|     clearance_level: 'confidential'
|   }
| ]
soc_lite> db.adminCommand({ listDatabases: 1 })
|
| {
|   databases: [ { name: 'soc_lite', sizeOnDisk: Long('155648'), empty: false } ],
|   totalSize: Long('155648'),
|   totalSizeMb: Long('0'),
|   ok: 1
| }
soc_lite>

```

■ Checklist hardening complétée — authentification, RBAC, réseau, secrets

Conclusion — Bilan du Hardening

Contrôle	Avant	Après
Authentification	Désactivée — accès anonyme total	Activée — identifiants requis
Comptes	Aucun — droits implicites admin	3 comptes RBAC + 1 admin
Rôles	Pas de RBAC — tout autorisé	Moindre privilège par profil
Binding réseau	0.0.0.0:27017 (toutes interfaces)	127.0.0.1:27017 uniquement
Secrets	Hardcodés / absents	Variables d'environnement / .env

Cette séance a permis de mettre en œuvre les trois piliers du durcissement MongoDB : l'**authentification** (qui peut se connecter), le **RBAC** (ce que chaque compte peut faire), et le **contrôle réseau** (depuis où la connexion est possible). L'application du principe de *défense en profondeur* garantit qu'aucune couche seule ne constitue le seul rempart, conformément aux bonnes pratiques de cybersécurité pour la protection des données d'un SOC.